HRMS
Master Architecture and Table Plan

One document for all 13 modules: what each module does, which existing tables to use, which to modify, and the minimum set of new tables to create.

Databases: Product_DB.public (operations) and SOC_SaaS_V2.public (identity)
Design goal: fewest new tables, more columns on existing tables
Consolidates: Architecture Guide, Functional Guide, 13-Modules Guide, Technical Specification

Contents

Contents 2

1. Executive Summary 3

2. Design Decisions That Settle the Contradictions 4

3. Master Delta Matrix 5

Platform Layer: Approvals, Config, Audit, Notifications 6

Core Employee Information and Directory 7

Attendance, Biometric and Geofencing 8

Leave and Time-Off 9

Shift Planning and Scheduling 10

Payroll, Statutory Compliance and Payslips 11

Statutory Rates as Data 12

Recruitment and Applicant Tracking (ATS) 13

Employee Lifecycle, Onboarding, Offboarding and Assets 14

Performance Appraisal and OKR/KPI 15

Learning, Training and Certifications 16

Expense Claims and Travel Reimbursement 17

HR Helpdesk, Grievances and Ticketing 18

Employee Engagement, Polls, Surveys and Recognition 19

Executive HR Analytics and Reports 20

4. tab_common_master Group Catalogue 21

5. Where This Design Has Limits 22

1. Executive Summary

Your four documents describe the same HRMS but disagree on table names, counts and design. This guide replaces them. It uses the **Technical Specification** as the picture of what already exists, takes business rules from the Functional Guide, and takes feature depth from the 13-Modules Guide. The result is a plan that needs **20 new tables** (the Technical Specification alone listed 27 and still lacked approvals, audit, punch log, effective-dated statutory rates and a payroll run).

Measure	Count	Notes
Existing Product_DB tables used	13	branch, department, designation, employee, shift template, shift assignment, attendance, leave request, holidays, payroll, performance review, common master, announcements
Existing tables modified (ALTER)	12	Only new columns, constraints and indexes. No renames, no drops.
New tables to create	20	4 platform, 1 attendance, 1 leave, 2 employee, 3 payroll, 3 recruitment, 1 lifecycle, 1 asset, 1 performance, 1 expense, 1 helpdesk, 1 engagement
New reporting views (not tables)	10	Headcount, attrition, absenteeism, leave balance, payroll register, PF ECR, ESI, PT, Form 24Q, diversity
Total Product_DB tables after build	33	13 existing + 20 new

Why only 20 tables

- **One configuration table**: tab_common_master is extended and holds leave types, salary components, statutory rates, numbering prefixes, approval flows, SLAs, expense limits, courses, surveys and more, each as a row under its own master_group.
- **One request table and one approval-step table** serve WFH, shift swap, regularization, overtime, promotion, transfer, resignation, loans and requisitions.
- **One ledger per concern**: leave credits and debits in one table; every payslip line in one table.
- **Sensitive data in one table** (PAN, UAN, bank) so it can be encrypted and restricted by role.
- **Variable records in one table**: documents, dependents, nominees, education, experience, certifications, training, tax declarations.
- **Reports are views**, so they cost no tables.

Trade-off to accept

Fewer tables means some tables carry a type column and a jsonb column. That is fine when each type is validated by CHECK constraints and application schemas, and when every type has a partial index. Section 5 lists the four places where you may later want to split a table out if volume demands it.

2. Design Decisions That Settle the Contradictions

These decisions fix the conflicts between your four documents. Apply them before writing any DDL.

Topic	Conflict in the documents	Decision used in this guide
Designation	Architecture Guide adds a DESIGNATION master; Tech Spec already has tab_employee_role_master.	Reuse tab_employee_role_master (staff_role_id). Make branch_id and department_id nullable so a designation can be organisation-wide.
Master tables	tab_hrms_master (new) vs tab_common_master (existing).	Extend tab_common_master. Do not create a third master mechanism. Scope every HRMS row by app_id and a master_group starting with HRMS_.
Manager column	reports_to_id vs manager_id.	manager_id, with a trigger that blocks self-reference and loops.
Leave types	Master row vs tab_leave_type_master.	Rows in tab_common_master (group HRMS_LEAVE_TYPE) with a rules jsonb: quota, paid, accrual, carry-forward, half-day, sandwich rule.
Leave balance	Entitled/taken vs opening/accrued/availed.	No balance table. A ledger (tab_leave_ledger) plus a view gives opening, accrued, availed, adjusted, encashed and current balance, and can be rebuilt and audited.
Early leaving	early_leaving_minutes vs early_exit_minutes.	early_exit_minutes.
Requests	One generic table vs separate swap and regularization tables.	One tab_hrms_request with request_type. Each type has its own CHECK and partial index.
Payroll lines	tab_payroll_breakdown vs component master plus item detail.	Components in tab_common_master; lines in tab_payroll_item_detail with an employer-contribution item type and a rate snapshot.
Bank and KYC	On employee master vs separate table.	Separate tab_employee_statutory. PAN and account number encrypted; Aadhaar stored as last four digits only.
Identifier types	Existing shift_id, leave_id, attendance_id, payroll_id are uuid; approved_by and reviewer_id are uuid; new tables use integer employee_id.	Keep existing uuid keys. All new people references are integer employee_id. Add new integer approver columns on leave and performance tables and leave the old uuid columns for legacy data.
Time and units	timestamp vs timestamptz; overtime hours divided by 60.	New time columns are timestamptz. Overtime is stored in minutes everywhere; convert to hours only for pay.
fiscal_year_id	13-Modules Guide says every table; department master carries it.	Only on rows that belong to a year (ledger, expense claim, KPI goal). Not on master data such as departments.
Statutory rates	Hardcoded 12 percent and 15,000 cap in three documents.	Effective-dated rows in tab_common_master plus a snapshot on every payslip line.
Module list	Governance (Architecture/Functional) vs LMS, Engagement, Analytics (13-Modules/Tech Spec).	Use the 13-Modules numbering. Governance becomes a platform layer (Module 00) underneath all modules.

Before you run anything

Confirm tab_common_master is not shared with another product in the same database (for example the restaurant app). If it is, the safe alternative is to create a copy named tab_hrms_config with the same columns. That would make it 21 new tables and 11 altered tables; nothing else changes.

3. Master Delta Matrix

Every module on one page: what is reused, what is altered, what is new.

Module	Existing tables used	Altered	New tables
00 Platform	tab_common_master; v2: organisation, users, role, user-org map, product master	tab_common_master	tab_hrms_request, tab_hrms_approval_step, tab_hrms_audit_log, tab_hrms_notification
01 Employee	tab_employee_master, branch, department, designation	employee master, branch master, department master, designation master	tab_employee_statutory, tab_employee_record
02 Attendance	tab_employee_attendance, branch master	attendance, branch master (geofence)	tab_attendance_punch
03 Leave	tab_employee_leave_request, tab_branch_holidays	leave request, holidays	tab_leave_ledger
04 Shifts	tab_shift_template, tab_employee_shift_assignment	shift template	none (WFH and swaps use tab_hrms_request)
05 Payroll and statutory	tab_employee_payroll, common master, attendance, ledger	payroll	tab_payroll_run, tab_employee_salary, tab_payroll_item_detail
06 Recruitment	department, branch, request, common master	none	tab_job_posting, tab_job_applicant, tab_applicant_interview
07 Lifecycle, onboarding, assets	employee master, request, approval step	employee master (already counted)	tab_lifecycle_task, tab_asset
08 Performance	tab_employee_performance_review, common master	performance review	tab_employee_kpi_goal
09 Learning	tab_employee_record, common master	none	none
10 Expenses	common master, approval step, payroll item detail	none	tab_expense_claim
11 Helpdesk	common master (categories, SLA)	none	tab_helpdesk_ticket
12 Engagement	tab_announcements, announcement recipients, common master	announcements	tab_engagement_entry
13 Analytics	all of the above	none	none (10 views)

Build order

Phase	Scope	Outcome
0	Fixes on existing tables: unique constraints, status CHECKs, timestamptz, indexes, org_id on any table lacking it	Safe base
1	Platform layer and Module 01 (employee, KYC, records, org tree)	Approvals, audit, numbering, notifications work
2	Modules 02, 03, 04 (punch log, shifts, leave ledger, holidays)	Clean attendance and leave data for payroll
3	Module 05 (salary, payroll run, statutory, payslips, F and F)	Payroll in production after a parallel run
4	Modules 07 and 10 (lifecycle, assets, expenses)	Joiner and leaver flow, reimbursements in payroll
5	Modules 06, 08, 09, 11, 12, 13	Hiring, appraisals, learning, helpdesk, engagement, analytics

MODULE 00

Platform Layer: Approvals, Config, Audit, Notifications

What it does. Shared services that every other module depends on. It replaces the separate Governance module and adds what none of the four documents modelled: per-step approvals, delegation, escalation, a field-level audit log and notifications.

- Configurable approval flows (single level, manager then HR, chain of command) held as rows, not code
- Numbering prefixes with yearly reset (EMP-2026-0001, PAY-2026-0001, HR-TKT-0001)
- Custom field definitions that appear on the employee profile
- Audit of who changed salary, bank account, designation or status, and when
- Email, in-app and reminder queue (probation due, document expiry, warranty expiry, SLA breach)

A. EXISTING TABLES TO USE

Table	How this module uses it
`tab_common_master`	Becomes the HRMS configuration engine (see Section 4 for every master_group).
`SOC_SaaS_V2: tab_organisation_master, tab_users, tab_role, tab_user_organisation_map, tab_product_master`	Tenant, login, role per organisation. HRMS needs one row in tab_product_master; app_id in Product_DB equals that product_id. Cross-database user_id cannot be a real foreign key, so enforce it in the application.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)

Table	Columns and constraints to add
`tab_common_master`	parent_id int, effective_from date, effective_to date, rules jsonb, sort_order int, is_active boolean. Unique (org_id, app_id, master_group, master_code, effective_from). Index on (org_id, app_id, master_group, is_active).

C. NEW TABLES TO CREATE

Table	Key columns (all new tables also carry org_id, app_id, created_at)	Purpose
`tab_hrms_request`	request_id, request_no, request_type, employee_id, related_employee_id, from_date, to_date, effective_date, payload jsonb, old_value jsonb, new_value jsonb, reason, status, current_level, total_levels, decided_at	One table for every approval-driven request and for career events
`tab_hrms_approval_step`	step_id, entity_type, entity_id (text, so uuid or int), level_no, approver_id, delegated_from_id, status, acted_at, comment, due_at, escalated_to_id	Who approved at which level, delegation, escalation
`tab_hrms_audit_log`	log_id bigint, table_name, row_id, employee_id, field_name, old_value, new_value, action, changed_by, changed_at timestamptz, ip_address	Field-level history; partition monthly
`tab_hrms_notification`	notif_id, user_id, employee_id, channel, template_code, subject, body, ref_type, ref_id, status, scheduled_at, sent_at, read_at	Outbox and reminders

Rules to enforce

- request_type values: WFH, SHIFT_SWAP, REGULARIZATION, OVERTIME, COMP_OFF, PROBATION_CONFIRM, PROMOTION, TRANSFER, SALARY_REVISION, RESIGNATION, TERMINATION, LOAN_ADVANCE, JOB_REQUISITION, OFFER, DOC_REQUEST. Add a CHECK on request_type and a CHECK that the required payload keys exist for each type.
- Approved career requests (promotion, transfer, salary revision) apply their new_value to the employee row on effective_date through a nightly job; the request row plus the audit log form the career history. No separate lifecycle ledger is needed.
- Prefix counters live in the HRMS_PREFIX row. Increment inside the same transaction with SELECT ... FOR UPDATE so two users never get the same number. Yearly reset is a field in rules.
- Delegation: when an approver has approved leave covering the request dates, the step is routed to the delegate. Escalation: a job moves overdue steps to the next level.
- Enable row-level security on every table with org_id; set org_id from the session, never from the client.

MODULE 01

Core Employee Information and Directory

What it does. Single source of truth for every person: profile, job details, reporting line, statutory identity, bank account, and documents.

- Employee directory with filters by branch, department, designation and status
- 360-degree profile: personal, contact, job, reporting, statutory, bank, documents
- Interactive org chart built from manager_id
- Document vault with verification status and expiry dates
- Emergency contacts, dependents, nominees, education and previous experience

A. EXISTING TABLES TO USE

Table	How this module uses it
tab_employee_master	Core employee row. Already holds code, name, contact, gender, DOB, join date, type, status, branch, department, staff_role_id.
tab_branch_master, tab_department_master, tab_employee_role_master	Work location, department tree, designation.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)

Table	Columns and constraints to add
tab_employee_master	manager_id int FK to itself (trigger blocks self and loops); grade_code, cost_center_code, blood_group, marital_status, nationality, personal_email, alternate_phone, permanent_address, probation_end_date, confirmation_date, resignation_date, last_working_date, notice_period_days, default_shift_id uuid, custom_fields jsonb. CHECK on status (ACTIVE, PROBATION, NOTICE, TERMINATED, ABSCONDED). Unique (org_id, app_id, employee_code).
tab_department_master	parent_department_id, head_employee_id. Remove fiscal_year_id usage from master data.
tab_employee_role_master	grade_code, level_no; make branch_id and department_id nullable.
tab_branch_master	state_code (needed for Professional Tax), latitude, longitude, geofence_radius_m.

C. NEW TABLES TO CREATE

Table	Key columns (all new tables also carry org_id, app_id, created_at)	Purpose
tab_employee_statutory	employee_id (unique), pan_no_enc, aadhaar_last4, uan_no, pf_no, esi_no, pf_applicable, esi_applicable, pt_state_code, tax_regime, bank_name, bank_branch, account_no_enc, ifsc_code, account_type, kyc_verified_at	Encrypted KYC and bank, one row per employee; bank changes tracked by the audit log
tab_employee_record	record_id, employee_id, record_type, title, details jsonb, file_url, issue_date, expiry_date, due_date, completed_on, score, status, verified_by, verified_at	Documents, family, education, experience, certifications, training, tax declarations, policy acknowledgments

Rules to enforce

- record_type values: DOCUMENT, ID_PROOF, EMERGENCY_CONTACT, DEPENDENT, NOMINEE, EDUCATION, EXPERIENCE, SKILL, CERTIFICATION, TRAINING, TAX_DECLARATION, POLICY_ACK. Add a partial index per heavily used type.
- Do not store full Aadhaar unless you have a legal reason. Encrypt PAN and account number at column level and grant tab_employee_statutory only to HR and payroll roles.
- Managers must not see salary or statutory data; ESS users see only rows where employee_id equals their own.
- Employee code comes from the HRMS_PREFIX setting; a new hire from recruitment inserts the employee row and the onboarding tasks in one transaction.

MODULE 02

Attendance, Biometric and Geofencing

What it does. Captures punches from biometric devices, the web portal and mobile GPS, then builds one verified daily attendance row per employee with late, early exit and overtime.

- Many punches per day (breaks, multiple entries) kept as a raw log
- GPS geofence check against the branch boundary
- Late minutes, early exit minutes and overtime minutes from the assigned shift
- Regularization of missed punches through the shared approval flow
- Overnight shifts, weekly offs, auto-absent marking

A. EXISTING TABLES TO USE	
Table	**How this module uses it**
tab_employee_attendance	Daily summary row (check-in, check-out, status, hours worked).
tab_branch_master	Geofence centre and radius (columns added in Module 01).
tab_shift_template, tab_employee_shift_assignment	Shift start, end, grace and thresholds for late and overtime.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)	
Table	**Columns and constraints to add**
tab_employee_attendance	shift_id uuid, shift_date date, in_latitude, in_longitude, out_latitude, out_longitude, check_in_device, check_out_device, in_ip, out_ip, late_minutes, early_exit_minutes, break_minutes, overtime_minutes, is_regularized, regularization_reason, regularization_request_id. Convert check_in_time and check_out_time to timestamptz. Unique (org_id, employee_id, attendance_date). CHECK on status (PRESENT, ABSENT, HALF_DAY, LATE, ON_LEAVE, HOLIDAY, WEEK_OFF, WFH). Plan monthly partitioning.

C. NEW TABLES TO CREATE		
Table	**Key columns (all new tables also carry org_id, app_id, created_at)**	**Purpose**
tab_attendance_punch	punch_id bigint, employee_id, punch_time timestamptz, direction (IN, OUT), source (BIOMETRIC, WEB, MOBILE), device_code, latitude, longitude, ip_address, is_valid, attendance_id	Raw punch log; the daily row is built from it

Rules to enforce

- Overtime is stored in minutes. Overtime minutes = max(0, worked minutes - standard shift minutes). Convert to hours only when paying.
- For overnight shifts, attendance_date is the date the shift started; shift_id decides the expected end on the next day.
- Punch outside the geofence and without an approved WFH request is saved with is_valid = false and flagged for review.
- Once the payroll run for the month is locked, block edits to attendance for that month.
- Device registry (device_code, branch, type) lives in tab_common_master group HRMS_DEVICE.

MODULE 03

Leave and Time-Off

What it does. Policy-driven leave with balances that can be audited, half-day support, holiday exclusion and multi-level approval.

- Leave types with quota, paid or unpaid, accrual, carry-forward, encashment, half-day and sandwich rules
- Automatic monthly or annual accrual and year-end carry-forward or lapse
- Half-day first or second half, medical certificate attachment after a threshold
- Optional and restricted holidays in the branch calendar
- Compensatory off earned through approved overtime or holiday work

A. EXISTING TABLES TO USE

Table	How this module uses it
tab_employee_leave_request	Application row (dates, days, reason, status).
tab_branch_holidays	Branch holiday calendar; excluded from leave day count.
tab_common_master	Leave types in group HRMS_LEAVE_TYPE.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)

Table	Columns and constraints to add
tab_employee_leave_request	leave_type_id int (FK to tab_common_master), is_half_day, half_day_period, document_proof_url, rejection_reason, approved_by_employee_id int, current_level, fiscal_year_id. Keep the old leave_type text and uuid approved_by for legacy rows. CHECK on status (DRAFT, SUBMITTED, PENDING, APPROVED, REJECTED, CANCELLED).
tab_branch_holidays	holiday_type (PUBLIC, OPTIONAL, RESTRICTED), is_optional_quota_counted. Unique (org_id, branch_id, holiday_date).

C. NEW TABLES TO CREATE

Table	Key columns (all new tables also carry org_id, app_id, created_at)	Purpose
tab_leave_ledger	ledger_id bigint, employee_id, leave_type_id, fiscal_year_id, entry_type (OPENING, ACCRUAL, AVAIL, ADJUST, ENCASH, LAPSE, CARRY_FWD), entry_date, days numeric(5,2) signed, leave_id uuid, remarks	Every credit and debit; replaces a balance table

Rules to enforce

- Balance = sum of ledger days per employee, leave type and fiscal year. Expose it through the view v_leave_balance (opening, accrued, availed, adjusted, encashed, current). Do not keep a separate counter that can drift.
- On approval insert an AVAIL row with negative days; on cancellation insert the reversing row. Never update or delete ledger rows.
- Pro-rata accrual for new joiners uses configurable accrual start date, fiscal year start and rounding rule from the leave type rules, not a fixed calendar-year formula.
- Block leave that overlaps an existing approved leave, and block edits once payroll for that month is locked.

MODULE 04

Shift Planning and Scheduling

What it does. Shift templates, weekly rosters, night shifts that cross midnight, and peer shift swaps.

- Morning, general and night shift templates with break and grace
- Roster by employee, department or day of week with effective dates
- Half-day and full-day minimum hours; weekly-off days; overtime multiplier
- Shift swap between two employees approved by the manager

A. EXISTING TABLES TO USE	
Table	**How this module uses it**
tab_shift_template	Template master.
tab_employee_shift_assignment	Roster rows (employee, shift, day of week, effective dates). No change needed.
tab_hrms_request	Shift swaps (SHIFT_SWAP) and WFH (WFH) requests.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)	
Table	**Columns and constraints to add**
tab_shift_template	grace_late_minutes, half_day_min_minutes, full_day_min_minutes, is_night_shift, weekly_off_days varchar, ot_multiplier numeric(3,2). Ignore consult_duration_minutes (left over from another product).

C. NEW TABLES TO CREATE

No new tables.

Rules to enforce

- Swap: request_type = SHIFT_SWAP with employee_id, related_employee_id and from_date; both the colleague and the manager must approve (two approval steps) before the roster is updated.
- WFH: an approved WFH request whitelists the employee from office IP and geofence checks for the approved dates.
- Convert shift_id usage to uuid everywhere (matches the existing table); do not add integer shift_id columns.

MODULE 05

Payroll, Statutory Compliance and Payslips

What it does. Monthly gross-to-net engine with effective-dated salary, attendance-driven payable days, statutory deductions from dated rate rows, adjustments, F and F and bank file.

- Salary structure with revisions and history
- Payroll run lifecycle: draft, computed, approved, locked, paid
- PF, EDLI, EPF admin charge, ESI, Professional Tax, LWF and TDS from effective-dated rates
- Loans, advances, arrears, bonus, leave encashment, full and final settlement
- PDF payslip and bank transfer file (NEFT or RTGS CSV)

A. EXISTING TABLES TO USE	
Table	**How this module uses it**
tab_employee_payroll	One row per employee per run (payslip header).
tab_common_master	Groups HRMS_SALARY_COMP, HRMS_STAT_PF, HRMS_STAT_ESI, HRMS_STAT_PT, HRMS_STAT_LWF, HRMS_TAX_SLAB.
tab_employee_attendance, tab_leave_ledger, tab_expense_claim, tab_hrms_request	Inputs: payable days, overtime, unpaid leave, reimbursements, loans.
tab_employee_statutory	Applicability flags, UAN, tax regime, state for PT.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)	
Table	**Columns and constraints to add**
tab_employee_payroll	run_id, pay_period date (first of month; keep old pay_period_month text), payslip_no, total_days, payable_days, lop_days, gross_earnings, total_deductions, reimbursements, net_pay, employer_contribution, overtime_pay, payment_method, payslip_url, status (DRAFT, COMPUTED, APPROVED, LOCKED, PAID). Make money columns numeric(15,2). Unique (org_id, employee_id, pay_period, run_id).

C. NEW TABLES TO CREATE		
Table	**Key columns (all new tables also carry org_id, app_id, created_at)**	**Purpose**
tab_payroll_run	run_id, run_type (REGULAR, SUPPLEMENTARY, BONUS, FNF), pay_period, cutoff_date, status, employee_count, total_gross, total_net, computed_at, approved_by, locked_at, paid_at	Controls lock and approval of the month
tab_employee_salary	salary_id, employee_id, effective_from, effective_to, annual_ctc, monthly_gross, components jsonb [{code, amount}], revision_request_id, status	Effective-dated CTC with history
tab_payroll_item_detail	item_id bigint, payroll_id (nullable until computed), employee_id, pay_period, component_id (to common master), item_type (EARNING, DEDUCTION, EMPLOYER_CONTRIB, REIMBURSEMENT, ADJUSTMENT), source (SYSTEM, MANUAL, LOAN, ARREAR, BONUS, ENCASH, FNF), amount, rate_snapshot jsonb, remarks	Every payslip line and every one-time input

Rules to enforce

- Calculation order: (1) lock attendance and leave; (2) payable days = days in month minus LOP, day basis (calendar, 26 or 30) set by policy; (3) pick the salary row valid on each day and split a month where a revision starts mid-month; (4) earned component = amount x payable days / month days; (5) add overtime pay = overtime minutes / 60 x hourly rate x multiplier; (6) work out statutory wages; (7) PF, ESI, PT; (8) TDS from declaration and projection; (9) add loans, arrears, bonus, encashment lines; (10) reimbursements as a separate non-taxable line; (11) Net = Gross - Deductions + Reimbursements.
- LOP is not a deduction line. It reduces payable days and therefore the earned amounts, which changes PF and ESI wages.
- Reimbursements never enter PF, ESI or TDS bases.
- Write the rate used on each line into rate_snapshot so old payslips never change when a rate changes.
- Unique one payroll row per employee per period per run; locked runs reject any edit to attendance, leave or salary for that month.

Statutory Rates as Data

Never hardcode a rate or ceiling. Store each as a dated row in tab_common_master, and pick the row valid on the day being calculated. A live example is the PF wage ceiling, which changed on 17 September 2026.

master_group / code	effective_from to effective_to	rules (jsonb)
HRMS_STAT_PF / PF_CEILING	2026-05-29 to 2026-09-16	{"wage_ceiling":15000,"employee_pct":12,"employer_epf_pct":3.67,"employer_eps_pct":8.33}
HRMS_STAT_PF / PF_CEILING	2026-09-17 onward	{"wage_ceiling":25000, ...same percentages}
HRMS_STAT_PF / PF_ADMIN	as notified	{"edli_pct":0.5,"admin_pct":0.5} (confirm current rates with your CA)
HRMS_STAT_ESI / ESI	as notified	{"wage_ceiling":21000,"employee_pct":0.75,"employer_pct":3.25}
HRMS_STAT_PT / PT_	per state	{"slabs":[{"from":0,"to":12000,"amount":0}, ...]}
HRMS_TAX_SLAB / OLD, NEW	per financial year	{"slabs":[...], "std_deduction":...}

- **September 2026 is a split month.** Published guidance says the PF wage month splits: 1 to 16 September on the 15,000 ceiling and 17 to 30 September on the 25,000 ceiling, reported in a single ECR. Sources differ on edge cases (newly covered members, EPS membership), so have your CA or payroll consultant confirm the exact rule before go-live. The ESI ceiling of 21,000 is unchanged.
- The four labour codes took effect on 21 November 2025. They require basic pay, DA and retaining allowance to be at least 50 percent of total remuneration, with excess allowances treated as wages. The engine therefore needs a **statutory wages** step before PF and ESI, not just basic. Confirm the final rule with your consultant.
- Professional Tax needs the state of the work location, which is why tab_branch_master gets state_code.
- Exports (PF ECR, ESI return, PT return, Form 24Q) are views over tab_payroll_item_detail and tab_employee_statutory.

MODULE 06

Recruitment and Applicant Tracking (ATS)

What it does. Job requisitions, candidate pipeline, interview scorecards, offers and one-click conversion to an employee.

- Requisition approval, job posting with openings and salary band
- Candidate pipeline: Applied, Screened, Technical, HR Interview, Offered, Hired, Rejected
- Several interviewers per candidate, each with rating and feedback
- Offer with approval, then conversion to employee and onboarding tasks
- Stage history for time-to-hire reports

A. EXISTING TABLES TO USE

Table	How this module uses it
`tab_department_master, tab_branch_master`	Job location and department.
`tab_hrms_request, tab_hrms_approval_step`	Requisition (JOB_REQUISITION) and offer (OFFER) approvals.
`tab_common_master`	Stage list (HRMS_JOB_STAGE), letter templates (HRMS_DOC_TEMPLATE).

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)

No changes.

C. NEW TABLES TO CREATE

Table	Key columns (all new tables also carry org_id, app_id, created_at)	Purpose
`tab_job_posting`	job_id, job_code, title, department_id, branch_id, requisition_request_id, openings, min_experience, salary_min, salary_max, description, closing_date, status (OPEN, PAUSED, CLOSED)	Open vacancies
`tab_job_applicant`	applicant_id, job_id, full_name, email, phone, resume_url, source, referred_by_employee_id, current_ctc, expected_ctc, stage, stage_history jsonb, offer jsonb {ctc, join_date, status, letter_url}, overall_score, converted_employee_id	Candidate, stage history and offer
`tab_applicant_interview`	interview_id, applicant_id, round_name, interviewer_id, scheduled_at, rating, feedback, recommendation, status	One row per interviewer per round

Rules to enforce

- When stage becomes HIRED, insert the employee, copy name, email, phone and resume into tab_employee_record, set converted_employee_id, and create onboarding tasks in one transaction.
- Unique (org_id, job_id, email) to stop duplicate applications.
- Offer letters are generated from an HRMS_DOC_TEMPLATE row; the letter URL is stored on the applicant offer.

MODULE 07

Employee Lifecycle, Onboarding, Offboarding and Assets

What it does. Probation, confirmation, promotion, transfer and resignation; joiner and leaver checklists; company asset custody; clearance before final settlement.

- Joiner checklist (offer signed, accounts created, induction)
- Probation reminders at 60 and 90 days; confirm, extend or exit
- Resignation, notice countdown, exit interview, no-dues clearance across HR, IT and Finance
- Asset register with serial number, issue acknowledgment and return condition
- Full and final settlement blocked until clearance is complete

A. EXISTING TABLES TO USE

Table	How this module uses it
tab_employee_master	Dates and status (columns added in Module 01).
tab_hrms_request	PROBATION_CONFIRM, PROMOTION, TRANSFER, RESIGNATION, TERMINATION requests; they double as the career history.
tab_payroll_run	FNF runs.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)

No changes.

C. NEW TABLES TO CREATE

Table	Key columns (all new tables also carry org_id, app_id, created_at)	Purpose
tab_lifecycle_task	task_id, employee_id, lifecycle_type (ONBOARDING, OFFBOARDING, PROBATION), task_name, owner_department (HR, IT, FINANCE, MANAGER), assigned_to, due_date, status, completed_at, completed_by, is_blocking, details jsonb	Checklists, clearance, exit interview
tab_asset	asset_id, asset_tag, category_code, asset_name, brand_model, serial_number, purchase_date, warranty_expiry_date, status (IN_STOCK, ASSIGNED, REPAIR, RETIRED), assigned_employee_id, assigned_on, issue_condition, ack_at, expected_return, returned_on, return_condition	Asset master and current custody; history from the audit log

Rules to enforce

- Unique (org_id, serial_number) on tab_asset. Every change of assigned_employee_id is recorded in tab_hrms_audit_log, which gives the issue and return history.
- A FNF payroll run cannot be approved while the employee has any lifecycle task with is_blocking = true and status not DONE, or any asset still ASSIGNED.
- Notice shortfall recovery and leave encashment are inserted as payroll item lines with source FNF.
- Probation and warranty reminders are rows in tab_hrms_notification created by a daily job.

MODULE 08

Performance Appraisal and OKR/KPI

What it does. Goals, progress tracking and review cycles with self, manager, peer and HR ratings.

- Goals and key results with weightage and progress
- Appraisal cycles (annual, mid-year, quarterly)
- 360-degree: one review row per rater
- HR calibration and final rating
- Promotion and increment recommendation feeding a SALARY_REVISION request

A. EXISTING TABLES TO USE

Table	How this module uses it
tab_employee_performance_review	Review rows.
tab_common_master	Cycles in group HRMS_PERF_CYCLE (name, start, end, status).

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)

Table	Columns and constraints to add
tab_employee_performance_review	cycle_id (to common master), reviewer_type (SELF, MANAGER, PEER, HR, FINAL), reviewer_employee_id int (keep old uuid reviewer_id for legacy), competency_scores jsonb, final_rating, promotion_recommended, increment_recommended_pct, status. Unique (org_id, employee_id, cycle_id, reviewer_type, reviewer_employee_id).

C. NEW TABLES TO CREATE

Table	Key columns (all new tables also carry org_id, app_id, created_at)	Purpose
tab_employee_kpi_goal	goal_id, employee_id, cycle_id, parent_goal_id, goal_title, goal_type, unit, target_value, achieved_value, weightage_percent, progress_percent, due_date, manager_rating, status, fiscal_year_id	Goals and OKRs

Rules to enforce

- Multiple raters are simply multiple rows per employee per cycle, so no separate 360 tables are needed.
- A manager sees reviews for their reports only; peer ratings are hidden from the employee unless policy allows.

MODULE 09

Learning, Training and Certifications

What it does. Course catalogue, mandatory compliance training, progress and certificates with expiry reminders.

- Course catalogue with trainer, dates and mandatory flag
- Automatic assignment of mandatory courses to new joiners
- Completion, score and certificate
- Expiry reminders for certifications

A. EXISTING TABLES TO USE

Table	How this module uses it
`tab_common_master`	Courses in group HRMS_COURSE (trainer, dates, is_mandatory, target department).
`tab_employee_record`	Enrollment as record_type TRAINING or CERTIFICATION with status, due_date, completed_on, score, expiry_date, file_url.
`tab_hrms_notification`	Deadline and expiry reminders.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)

No changes.

C. NEW TABLES TO CREATE

No new tables.

Rules to enforce

- Enrollment status values: ENROLLED, IN_PROGRESS, COMPLETED, DROPPED, EXPIRED.
- Unique (employee_id, record_type, title, due_date) for training rows to stop duplicate assignment.

MODULE 10

Expense Claims and Travel Reimbursement

What it does. Itemised claims with receipts, policy limits, multi-level approval and payout through payroll or direct transfer.

- Claim with several line items and receipts
- Category limits per item and per day
- Manager approval, then Finance approval
- Advances adjusted against the claim
- Approved claims flow into the next payslip as a non-taxable reimbursement

A. EXISTING TABLES TO USE

Table	How this module uses it
tab_common_master	Categories and limits in group HRMS_EXPENSE_CAT.
tab_hrms_approval_step	Approval levels (entity_type EXPENSE).
tab_payroll_item_detail	Reimbursement line with source EXPENSE.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)

No changes.

C. NEW TABLES TO CREATE

Table	Key columns (all new tables also carry org_id, app_id, created_at)	Purpose
tab_expense_claim	claim_id, claim_no, employee_id, title, trip_from, trip_to, advance_amount, total_amount, approved_amount, currency, items jsonb [{category_code, date, amount, merchant, receipt_url}], status (DRAFT, SUBMITTED, APPROVED, REJECTED, PAID), payroll_id, paid_on, fiscal_year_id	Voucher and line items

Rules to enforce

- Each item is checked against its category limit when the claim is submitted.
- The items array is validated by the application and indexed with a GIN index if you need line-level reports. If you later need per-line approval, split items into a child table (see Section 5).

MODULE 11

HR Helpdesk, Grievances and Ticketing

What it does. Confidential, SLA-tracked tickets for payroll queries, leave corrections, policy questions and grievances.

- Ticket with category, priority and attachments
- Two-way conversation thread
- Confidential flag for POSH and other sensitive cases
- SLA due times for first response and resolution

A. EXISTING TABLES TO USE

Table	How this module uses it
tab_common_master	Categories (HRMS_TICKET_CAT) and SLA targets (HRMS_SLA).
tab_hrms_notification	Breach and reply alerts.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)

No changes.

C. NEW TABLES TO CREATE

Table	Key columns (all new tables also carry org_id, app_id, created_at)	Purpose
tab_helpdesk_ticket	ticket_id, ticket_no, employee_id, category_code, priority, subject, description, is_confidential, assigned_to, status (OPEN, IN_PROGRESS, RESOLVED, CLOSED), first_response_due, resolution_due, first_responded_at, resolved_at, resolution_notes, thread jsonb [{by, at, text, file_url}], attachments jsonb	Ticket with thread and SLA

Rules to enforce

- SLA targets as in the Functional Guide: Critical 2 hours response and 24 hours resolution; High 4 and 48; Medium and Low 12 to 24 hours response. Store them in HRMS_SLA rows, not in code.
- Confidential tickets are visible only to the named compliance officers; use a row-level policy on is_confidential.
- Append to thread in a single UPDATE so two replies never overwrite each other.

MODULE 12

Employee Engagement, Polls, Surveys and Recognition

What it does. Announcements with read receipts, anonymous surveys, polls and peer recognition.

- Announcements and policy notices with acknowledgment
- Surveys and pulse polls with optional anonymity
- Kudos and badges between colleagues
- Birthday and work-anniversary widgets from employee master

A. EXISTING TABLES TO USE	
Table	**How this module uses it**
`tab_announcements, tab_announcement_recipients`	Broadcasts and read receipts (confirm the recipients table exists; it is named only in the 13-Modules Guide).
`tab_common_master`	Survey definitions (HRMS_SURVEY, questions in rules) and badge types (HRMS_BADGE).
`tab_employee_record`	POLICY_ACK rows when a policy needs a signed acknowledgment.

B. EXISTING TABLES TO MODIFY (ALTER TABLE, add columns only)	
Table	**Columns and constraints to add**
`tab_announcements`	requires_ack boolean, expires_at, target_scope jsonb.

C. NEW TABLES TO CREATE		
Table	**Key columns (all new tables also carry org_id, app_id, created_at)**	**Purpose**
`tab_engagement_entry`	entry_id, entry_type (SURVEY_RESPONSE, POLL_VOTE, RECOGNITION), survey_id, from_employee_id (null when anonymous), to_employee_id, respondent_hash, rating, answers jsonb, message, badge_code	Survey answers, votes, kudos

Rules to enforce

- For anonymous surveys keep from_employee_id null and use respondent_hash only to stop duplicate answers; never store a way back to the person.

MODULE 13

Executive HR Analytics and Reports

What it does. Turns transactional data into headcount, attrition, absenteeism, payroll cost and statutory reports. It needs **no new tables**; everything is a view or materialized view refreshed nightly.

View	Built from	Used for
`v_headcount`	employee master, department, branch	Active headcount, probation vs confirmed, growth
`v_attrition`	employee master dates, RESIGNATION and TERMINATION requests	Monthly and annual turnover, exit reasons
`v_absenteeism`	attendance, leave ledger	Absence and late patterns by department
`v_leave_balance`	tab_leave_ledger	Balance per employee, type and year
`v_payroll_register`	payroll, item detail	Department-wise cost, overtime cost, variance
`v_pf_ecr, v_esi_return`	item detail, statutory	Monthly PF and ESI filings
`v_pt_return, v_form24q`	item detail, statutory	Professional Tax and TDS returns
`v_diversity`	employee master	Gender ratio and age bands (aggregates only)

Add indexes on (org_id, app_id, date) for the large tables and refresh materialized views after each payroll lock.

4. tab_common_master Group Catalogue

Every master_group below is a set of rows in the same table. Each has its own rules jsonb shape. Validate the shape in the application and with a CHECK on master_group.

master_group	Holds	Example rules
HRMS_LEAVE_TYPE	Leave policies	{"annual_quota":12,"paid":true,"accrual":"MONTHLY","carry_forward_max":6,"encashable":false,"half_day":true,"sandwich":false,"doc_after_days":2}
HRMS_SALARY_COMP	Salary heads	{"kind":"EARNING","calc":"PERCENT_OF","base":"BASIC","value":40,"taxable":true,"pf_wage":false,"esi_wage":true}
HRMS_STAT_PF, _ESI, _PT, _LWF	Statutory rates and slabs, effective-dated	See Module 05 statutory page
HRMS_TAX_SLAB	Income tax slabs by regime and year	{"regime":"NEW","slabs":[...]}
HRMS_PREFIX	Numbering	{"prefix":"EMP-","pattern":"{YYYY}-{SEQ:4}","reset":"YEARLY","next_seq":1}
HRMS_APPROVAL_FLOW	Approval matrix per module	{"module":"LEAVE","levels":[{"approver":"MANAGER"},{"approver":"HR"}],"escalate_after_hours":48}
HRMS_CUSTOM_FIELD	Custom employee fields	{"entity":"EMPLOYEE","type":"TEXT","required":false}
HRMS_EXPENSE_CAT	Expense categories and limits	{"limit_per_item":1500,"receipt_required_above":500}
HRMS_SLA	Helpdesk SLA by priority	{"priority":"CRITICAL","response_hours":2,"resolution_hours":24}
HRMS_TICKET_CAT, HRMS_ASSET_CAT, HRMS_DOC_TYPE	Plain lookups	{}
HRMS_DOC_TEMPLATE	Offer, appointment, experience letter templates	{"body_url":"...","placeholders":[...]}
HRMS_PERF_CYCLE	Appraisal cycles	{"start":"2026-04-01","end":"2027-03-31","status":"ACTIVE"}
HRMS_COURSE	Training catalogue	{"trainer":"...","mandatory":true,"start":"...","end":"..."}
HRMS_SURVEY, HRMS_BADGE	Survey definitions and award badges	{"anonymous":true,"questions":[...]}
HRMS_DEVICE	Biometric devices	{"branch_id":1,"type":"ZK","serial":"..."}
HRMS_GRADE, HRMS_COST_CENTER, HRMS_JOB_STAGE, HRMS_NOTIF_TEMPLATE	Remaining lookups and templates	{}

5. Where This Design Has Limits

Cutting tables moves some structure into jsonb and type columns. These are the places to watch. Split a table out only when you measure the problem, not before.

Table	Risk	Split into (when needed)
tab_expense_claim.items	No per-line approval or per-line payment; line-level reports need jsonb queries.	tab_expense_claim_item
tab_helpdesk_ticket.thread	Very long threads make the row large; no per-comment audit.	tab_helpdesk_ticket_comment
tab_job_applicant.stage_history and offer	Hard to query offer funnel at large hiring volume.	tab_applicant_stage_history
tab_employee_record	Large mixed table; some types (documents) grow quickly.	Move DOCUMENT rows to their own table

Guardrails that keep generic tables safe

- A CHECK constraint on every type or status column, with the allowed values listed in this guide.
- A partial index for each frequently queried type, for example on tab_hrms_request where request_type = REGULARIZATION.
- org_id and app_id on every new table, with row-level security; composite foreign keys where a reference must stay inside one tenant.
- Application-side jsonb schema validation per type; reject unknown keys.
- Monthly partitioning for tab_employee_attendance, tab_attendance_punch and tab_hrms_audit_log.
- Background jobs: accruals, auto-absent, payroll run, probation, warranty and document-expiry reminders, SLA escalation.
- Bulk import and export by Excel for employees, attendance and salary from day one.

Phase 0 fixes on the existing tables

- Check the live database before applying anything: run SELECT table_name, column_name, data_type FROM information_schema.columns WHERE table_schema = public AND table_name LIKE tab_%. Your ERD exports may have dropped unique and partial indexes, so confirm what already exists.
- Add unique constraints: one attendance row per employee per date; one payroll row per employee per period per run; one employee_code per tenant.
- Replace free varchar status columns with CHECK constraints.
- Change money columns to numeric(15,2) and timestamp columns to timestamptz where new logic depends on them.
- Add indexes on org_id, app_id, employee_id and date columns of the existing tables.

Payroll test cases to pass before go-live

- Employee on the PF ceiling boundary, and the September 2026 split month (1 to 16 on 15,000, 17 to 30 on 25,000).
- Employee crossing the ESI 21,000 threshold during the year.
- Mid-month joiner, mid-month leaver, mid-month salary revision.
- LOP days, half-day leave, overtime in minutes, overnight shift.
- Reimbursement paid with no effect on PF, ESI or TDS.
- F and F with notice shortfall, leave encashment and unreturned asset.
- A parallel run against your current payroll method for at least one full cycle.

Statutory rules change often. Have a payroll consultant or CA confirm the final PF, ESI, PT and TDS logic before go-live.